SIMATS ENGINEERING

ASSESSMENT TOOL 3: Mock Interview

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1522

Register Number	192324088
Name	Shaik Ashraf

COURSE OUTCOME COVERED

CO5: Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.

(BL5)

Dave's Taxonomy: Articulation (Level 4) Total Marks: 25

Question Count: 10

Code of Conduct

I Shaik Ashraf / 192324088 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____  _____

Assessment Tasks

Mock Interview

Answer the following interview-oriented questions clearly and concisely. Support your responses with architecture-level reasoning wherever appropriate.

1. Explain the difference between HDFS and traditional file systems.
2. Why is replication important in distributed storage systems?
3. How does MapReduce divide work between map and reduce phases?
4. What role does YARN play in large-scale Hadoop processing?
5. Differentiate NAS and SAN in an enterprise data environment.
6. What is the purpose of ETL in a big data pipeline?
7. How does Apache NiFi help in data integration workflows?

8. What are the key failure points in a Hadoop cluster and how would you handle them?

9. How would you design a secure data ingestion pipeline for a large organization?

10. How would you explain a scalable Hadoop-based processing system to a non-technical manager?

Question 1 – HDFS vs Traditional File Systems

Interview Objective

Provide a clear, concise and technically accurate interview response, with architecture-level reasoning where appropriate.

Answer

- HDFS is a distributed file system designed to store very large datasets across a cluster of commodity machines, whereas a traditional file system normally manages storage attached to a single computer or a smaller centralized storage environment.
- HDFS divides files into large blocks and distributes those blocks across multiple DataNodes. Traditional file systems generally keep file data within the storage system directly attached to the host.
- HDFS provides built-in replication and fault tolerance. If a DataNode fails, another replica can serve the data.
- HDFS is optimized for high-throughput sequential access and large-scale batch processing rather than low-latency access to many tiny files.
- The NameNode manages namespace and block metadata, while DataNodes store the actual blocks.

Key Points

- HDFS is a distributed file system designed to store very large datasets across a cluster of commodity machines, whereas a traditional file system normally manages storage attached to a single computer or a smaller centralized storage environment.
- HDFS divides files into large blocks and distributes those blocks across multiple DataNodes. Traditional file systems generally keep file data within the storage system directly attached to the host.
- HDFS provides built-in replication and fault tolerance. If a DataNode fails, another replica can serve the data.
- HDFS is optimized for high-throughput sequential access and large-scale batch processing rather than low-latency access to many tiny files.

Technical Comparison / Structure

Feature	HDFS	Traditional File System
Storage model	Distributed across multiple DataNodes	Usually local/centralized storage
Scalability	Horizontal scaling by adding nodes	Typically limited by host/storage system
Fault tolerance	Replication across DataNodes	Usually depends on underlying storage/backup
Metadata	Managed by NameNode	Managed by local/central file-system metadata

Typical workload	Large-scale big-data processing	General-purpose file operations

Practical Interview Insight

In an interview, emphasize that HDFS is optimized for distributed, high-throughput processing and fault tolerance rather than general-purpose low-latency file operations.

Interview Answer Flowchart

Client Application
↓
↓
↓
NameNode Resolves File Metadata
↓
↓
↓
HDFS Blocks
↓
↓
↓
Blocks Stored Across DataNodes
↓
↓
↓
Replicated Copies
↓
↓
↓
Parallel Data Access

Question 2 – Importance of Replication in Distributed Storage

Interview Objective

Provide a clear, concise and technically accurate interview response, with architecture-level reasoning where appropriate.

Answer

- Replication means maintaining multiple copies of data blocks on different storage nodes or failure domains.
- It improves availability because a failed node does not necessarily make the data inaccessible.
- It provides fault tolerance against hardware, disk, node, or sometimes rack-level failures.
- Replication can improve read throughput because clients may read from an available nearby replica.
- In HDFS, the replication factor determines how many copies of a block are maintained.
- Replication increases storage consumption, so the factor should be selected according to availability, durability, performance and cost requirements.

Key Points

- Replication means maintaining multiple copies of data blocks on different storage nodes or failure domains.
- It improves availability because a failed node does not necessarily make the data inaccessible.
- It provides fault tolerance against hardware, disk, node, or sometimes rack-level failures.
- Replication can improve read throughput because clients may read from an available nearby replica.

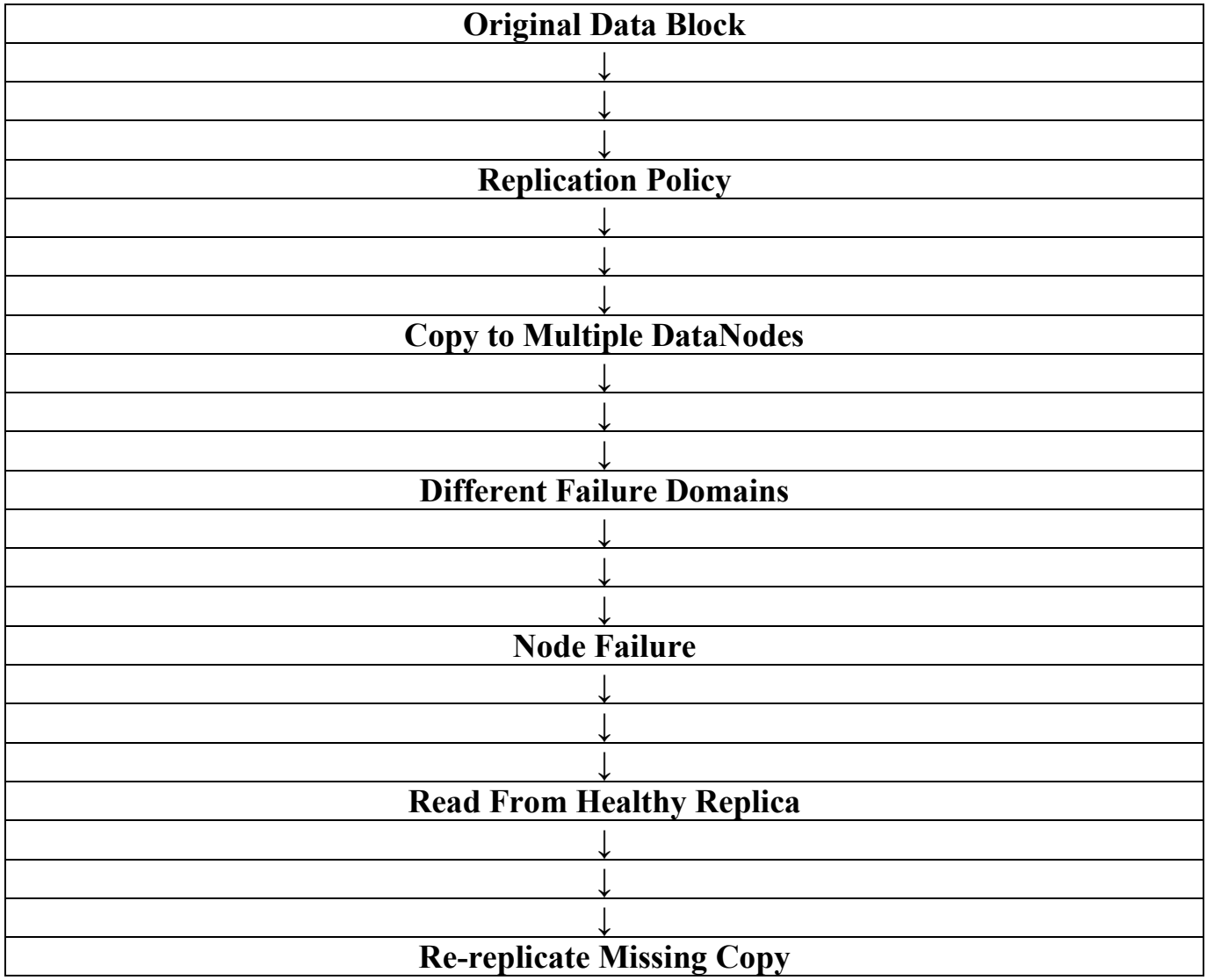
Technical Comparison / Structure

Aspect	Without Replication	With Replication
Node failure	May cause data unavailability	Another replica can serve the data
Availability	Lower	Higher
Storage cost	Lower	Higher
Read scalability	Limited	Can improve through multiple replicas
Recovery	May require external backup	HDFS can re-replicate missing copies

Practical Interview Insight

Replication is a trade-off: it improves availability and durability but consumes additional storage capacity.

Interview Answer Flowchart



Question 3 – MapReduce Map and Reduce Phases

Interview Objective

Provide a clear, concise and technically accurate interview response, with architecture-level reasoning where appropriate.

Answer

- MapReduce divides a batch-processing job into parallel stages so that large datasets can be processed across many machines.
- The Mapper reads input records, processes each record or group of records, and emits intermediate key-value pairs.
- The framework then performs partitioning, shuffle and sort so that values belonging to the same key are grouped for a Reducer.
- The Reducer receives each key and its associated values and performs aggregation, transformation or another final computation.
- The output is written to distributed storage such as HDFS.
- This model provides data-parallel processing and allows failed tasks to be re-executed.

Key Points

- MapReduce divides a batch-processing job into parallel stages so that large datasets can be processed across many machines.
- The Mapper reads input records, processes each record or group of records, and emits intermediate key-value pairs.
- The framework then performs partitioning, shuffle and sort so that values belonging to the same key are grouped for a Reducer.
- The Reducer receives each key and its associated values and performs aggregation, transformation or another final computation.

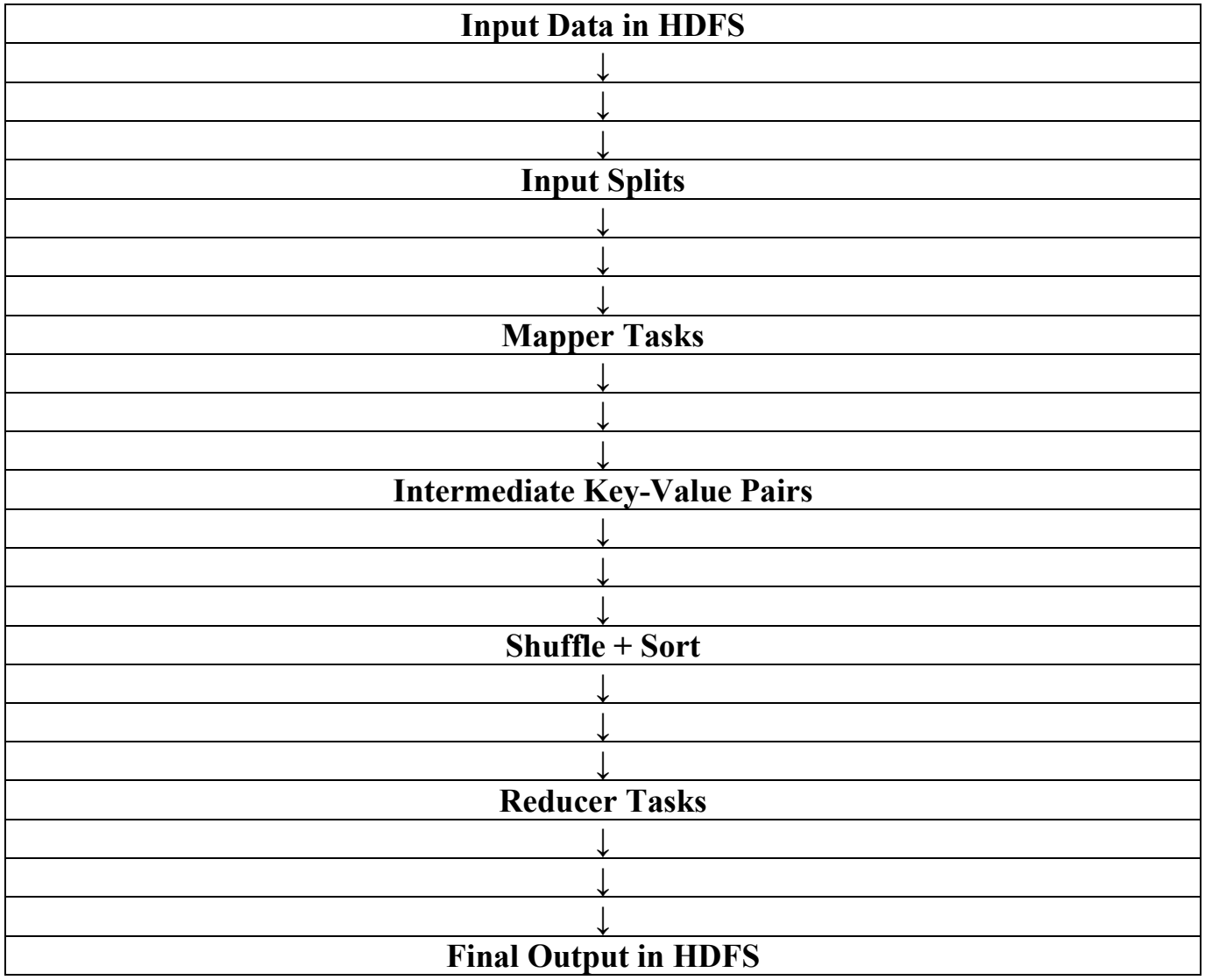
Technical Comparison / Structure

Stage	Responsibility	Typical Example
Input	Read HDFS blocks/splits	Read log records
Mapper	Transform records into key-value pairs	(word, 1)
Shuffle/Sort	Group values by key	All counts for a word
Reducer	Aggregate grouped values	Sum counts
Output	Write final results	Store result in HDFS

Practical Interview Insight

The important idea is that MapReduce converts a large computation into parallel map work followed by grouped reduce work, with shuffle/sort connecting the phases.

Interview Answer Flowchart


Input Data in HDFS
Input Splits
Mapper Tasks
Intermediate Key-Value Pairs
Shuffle + Sort
Reducer Tasks
Final Output in HDFS

Question 4 – Role of YARN in Hadoop Processing

Interview Objective

Provide a clear, concise and technically accurate interview response, with architecture-level reasoning where appropriate.

Answer

- YARN stands for Yet Another Resource Negotiator and provides resource management and application scheduling for Hadoop clusters.
- The ResourceManager is the central authority for allocating cluster resources and coordinating applications.
- NodeManagers run on worker nodes and manage containers and application resources on those nodes.
- Applications request resources through YARN, and YARN allocates containers based on the configured scheduling policy.
- YARN allows different processing frameworks and workloads to share the same cluster.
- Queue policies, capacity limits, priorities and monitoring help prevent resource starvation and improve utilization.

Key Points

- YARN stands for Yet Another Resource Negotiator and provides resource management and application scheduling for Hadoop clusters.
- The ResourceManager is the central authority for allocating cluster resources and coordinating applications.
- NodeManagers run on worker nodes and manage containers and application resources on those nodes.
- Applications request resources through YARN, and YARN allocates containers based on the configured scheduling policy.

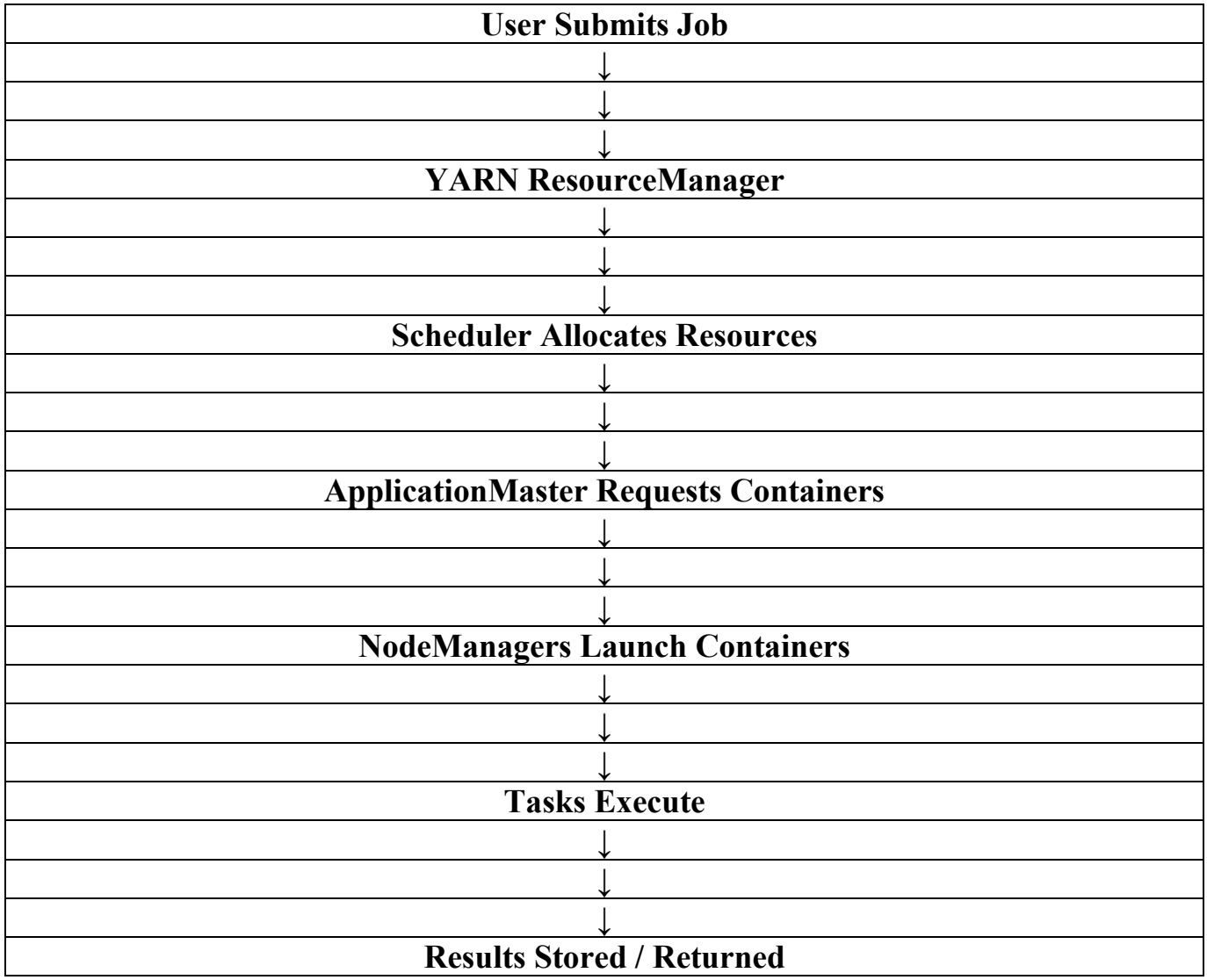
Technical Comparison / Structure

Component	Role
ResourceManager	Global resource allocation and scheduling
NodeManager	Manages resources and containers on each worker node
ApplicationMaster	Coordinates a particular application
Container	Allocated unit of CPU/memory resources
Scheduler	Determines how available resources are shared

Practical Interview Insight

YARN is the resource-management layer that enables different applications to share cluster resources using containers and scheduling policies.

Interview Answer Flowchart



Question 5 – NAS vs SAN

Interview Objective

Provide a clear, concise and technically accurate interview response, with architecture-level reasoning where appropriate.

Answer

- NAS provides file-level access over a network. Clients typically access shared files through network protocols and see a file-oriented namespace.
- SAN provides block-level storage to servers. The server generally manages the file system on the presented block devices.
- NAS is commonly convenient for shared files and collaborative access, while SAN is commonly used where applications need high-performance block storage.
- Both are centralized enterprise storage approaches, but they expose storage differently.
- The choice depends on workload, performance, sharing requirements, management model and cost.

Key Points

- NAS provides file-level access over a network. Clients typically access shared files through network protocols and see a file-oriented namespace.
- SAN provides block-level storage to servers. The server generally manages the file system on the presented block devices.
- NAS is commonly convenient for shared files and collaborative access, while SAN is commonly used where applications need high-performance block storage.
- Both are centralized enterprise storage approaches, but they expose storage differently.

Technical Comparison / Structure

Feature	NAS	SAN
Access	File-level	Block-level
Typical protocols	SMB/NFS	Fibre Channel, iSCSI and related technologies
Client view	Shared files/directories	Block devices/volumes
Common use	File sharing and shared documents	Databases, virtualization and high-performance applications
Management	File system often managed by NAS	Host/server commonly manages file system

Practical Interview Insight

The key distinction is file-level NAS versus block-level SAN; the correct choice depends on application and storage requirements.

Interview Answer Flowchart

Enterprise Applications
↓
↓
↓
Storage Requirement Identified
↓
↓
↓
File-Level Need? → NAS
↓
↓
↓
Block-Level Need? → SAN
↓
↓
↓
Provision Storage
↓
↓
↓
Connect Servers / Clients
↓
↓
↓
Monitor Performance and Capacity

Question 6 – Purpose of ETL in a Big Data Pipeline

Interview Objective

Provide a clear, concise and technically accurate interview response, with architecture-level reasoning where appropriate.

Answer

- ETL stands for Extract, Transform and Load.
- Extract collects data from operational databases, files, APIs, enterprise systems or other sources.
- Transform cleans, validates, standardizes and enriches data so that it can be used consistently for analytics.
- Load places the prepared data into a target system such as a data warehouse, data lake or analytics store.
- ETL improves data quality and creates a repeatable pipeline between source systems and analytical workloads.
- In large environments, ETL should include logging, validation, error handling, scheduling and lineage.

Key Points

- ETL stands for Extract, Transform and Load.
- Extract collects data from operational databases, files, APIs, enterprise systems or other sources.
- Transform cleans, validates, standardizes and enriches data so that it can be used consistently for analytics.
- Load places the prepared data into a target system such as a data warehouse, data lake or analytics store.

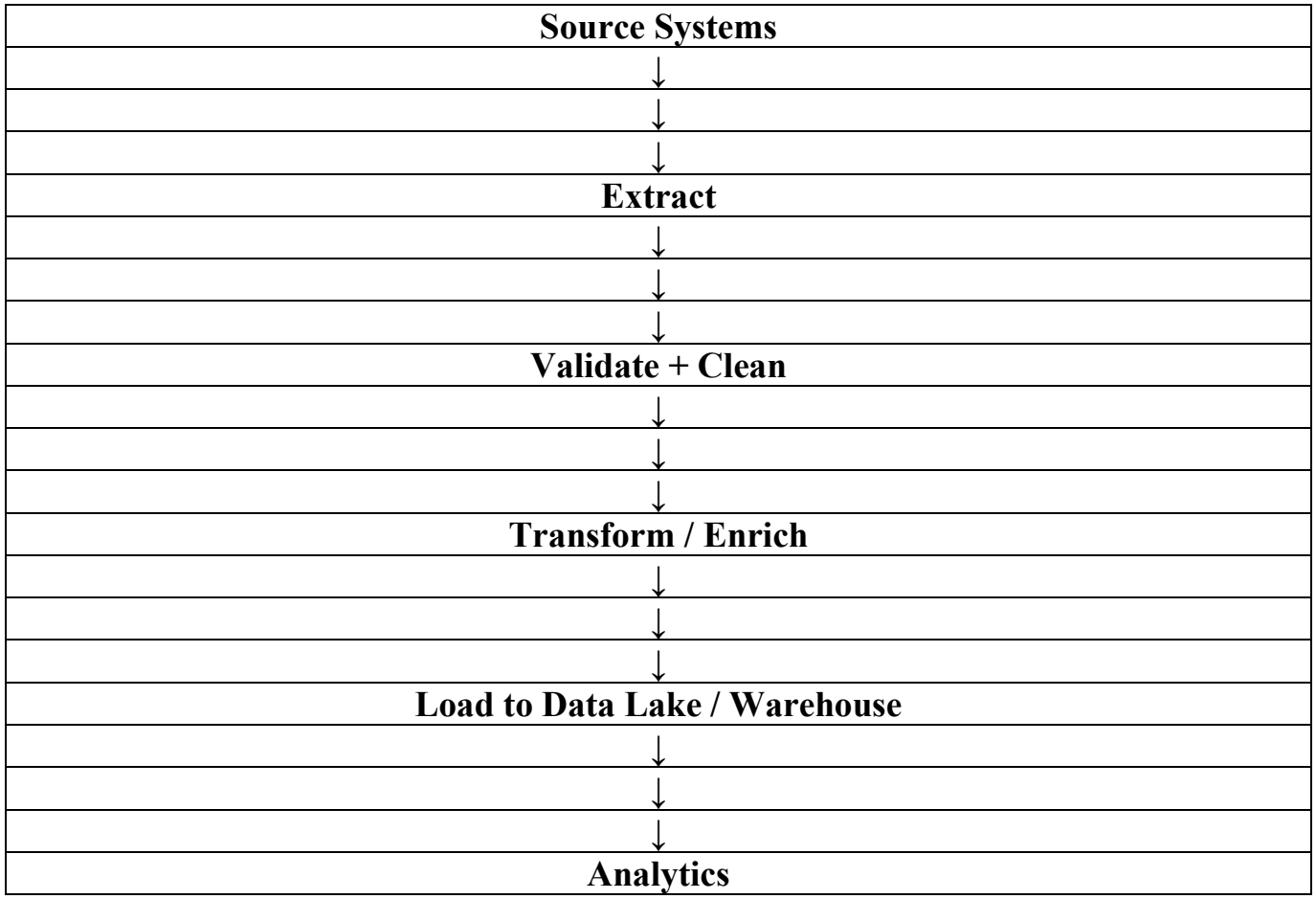
Technical Comparison / Structure

ETL Stage	Main Activity	Example
Extract	Collect source data	Read database tables or files
Transform	Clean and standardize	Handle nulls, formats and business rules
Validate	Check quality	Schema and business-rule validation
Load	Write target data	Load HDFS/data lake/warehouse
Monitor	Track pipeline health	Logs, metrics and failed records

Practical Interview Insight

ETL creates a controlled path from raw source data to clean, standardized analytical data, improving quality and repeatability.

Interview Answer Flowchart



Question 7 – Apache NiFi in Data Integration

Interview Objective

Provide a clear, concise and technically accurate interview response, with architecture-level reasoning where appropriate.

Answer

- Apache NiFi is a data-flow and integration platform used to move, route, transform and monitor data between systems.
- It provides processors that can ingest data from different sources, transform or validate it, and route it to destinations such as HDFS or databases.
- NiFi provides data provenance, which helps trace where a piece of data came from and how it moved through a flow.
- Flow-based configuration makes it useful for building visual ingestion pipelines with routing and error-handling paths.
- NiFi can support back pressure, prioritization and controlled flow so downstream systems are not overwhelmed.
- It can therefore act as an ingestion and integration layer in a Hadoop ecosystem.

Key Points

- Apache NiFi is a data-flow and integration platform used to move, route, transform and monitor data between systems.
- It provides processors that can ingest data from different sources, transform or validate it, and route it to destinations such as HDFS or databases.
- NiFi provides data provenance, which helps trace where a piece of data came from and how it moved through a flow.
- Flow-based configuration makes it useful for building visual ingestion pipelines with routing and error-handling paths.

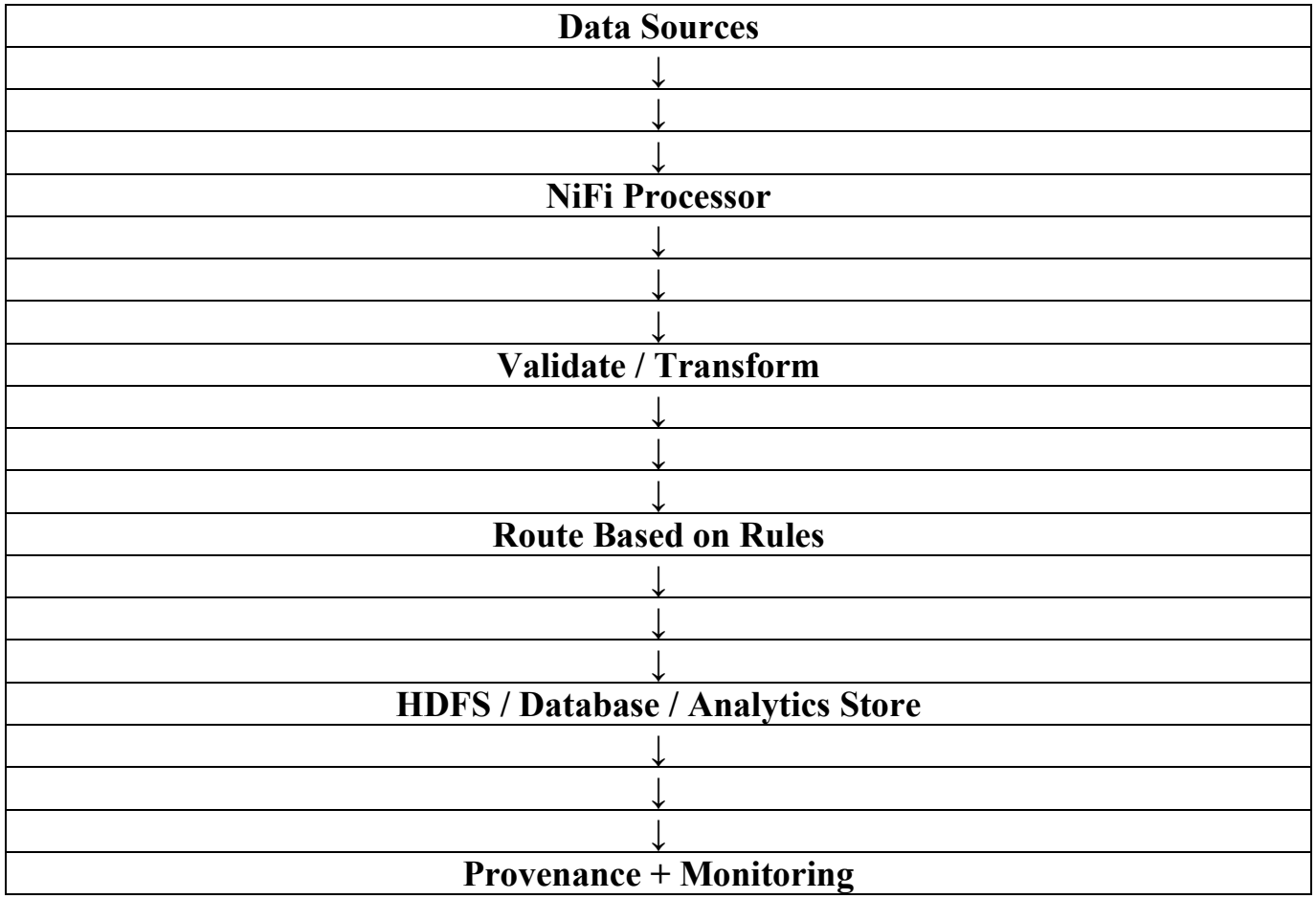
Technical Comparison / Structure

NiFi Capability	Purpose
Processors	Perform ingestion, transformation and routing
Connections	Move FlowFiles between processing stages
Provenance	Track data lineage and movement
Back Pressure	Prevent downstream overload
Routing	Send data based on rules or attributes
Monitoring	Observe flow status and failures

Practical Interview Insight

NiFi is particularly useful for visual flow management, routing, transformation, provenance and controlled data movement.

Interview Answer Flowchart



Question 8 – Hadoop Failure Points and Handling

Interview Objective

Provide a clear, concise and technically accurate interview response, with architecture-level reasoning where appropriate.

Answer

- Common failure points include DataNode failure, NameNode or metadata-service failure, disk failures, ResourceManager failure, network issues and application/task failures.
- DataNode failures are handled through replication and re-replication of missing blocks.
- NameNode high availability can use Active and Standby NameNodes so metadata service failure does not create a single point of failure.
- YARN ResourceManager high availability can provide resilient resource management.
- Task failures can often be handled through retry and re-execution mechanisms.
- Monitoring, alerts, rack-aware placement, capacity planning and tested recovery procedures are essential for operational resilience.

Key Points

- Common failure points include DataNode failure, NameNode or metadata-service failure, disk failures, ResourceManager failure, network issues and application/task failures.
- DataNode failures are handled through replication and re-replication of missing blocks.
- NameNode high availability can use Active and Standby NameNodes so metadata service failure does not create a single point of failure.
- YARN ResourceManager high availability can provide resilient resource management.

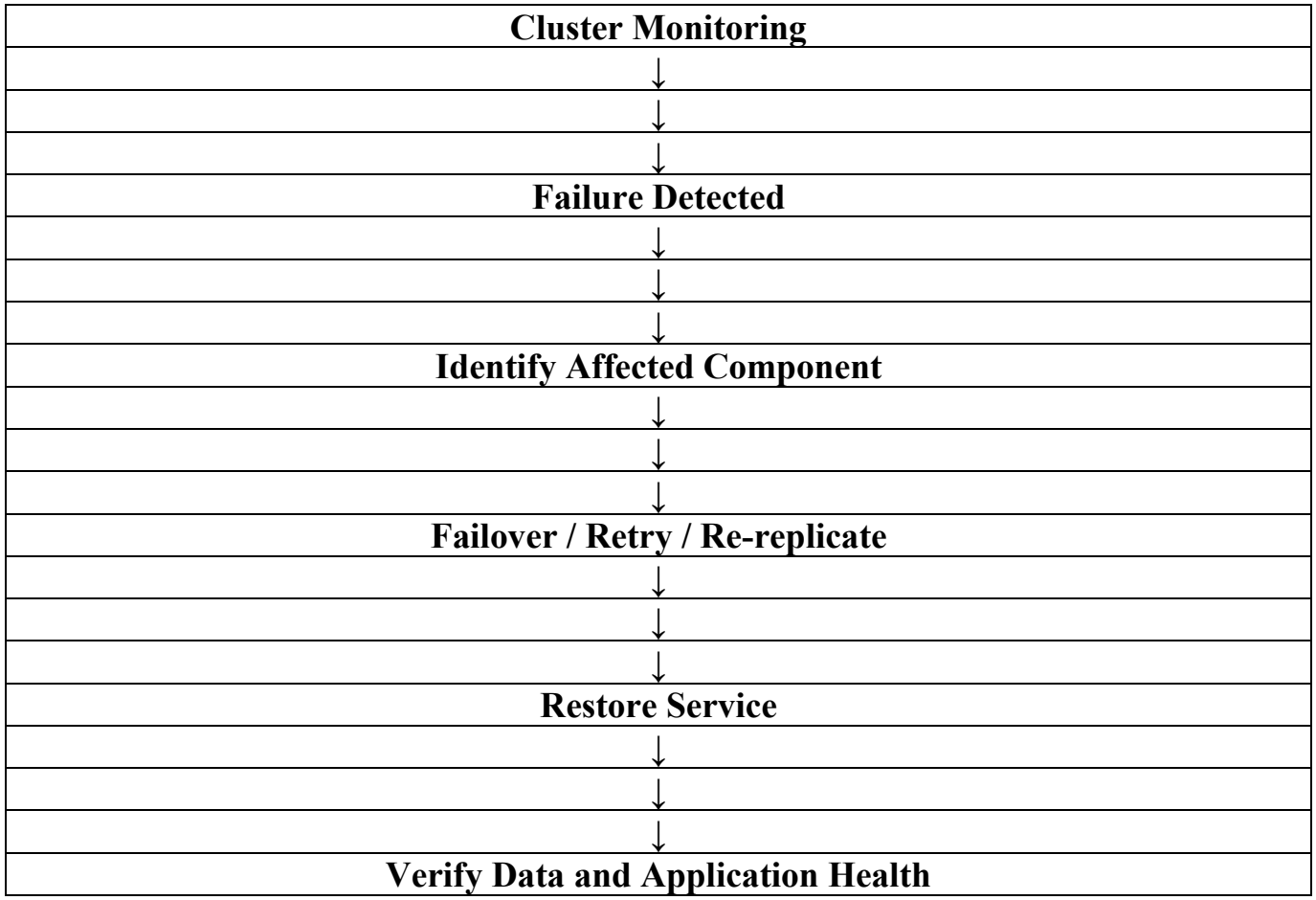
Technical Comparison / Structure

Failure Point	Impact	Handling Approach
DataNode	Some blocks may become unavailable	Replication + re-replication
NameNode	Namespace access can be disrupted	Active/Standby NameNode HA
Disk	Stored blocks can be lost/corrupted	Replication + health monitoring
ResourceManager	Scheduling may be affected	ResourceManager HA where required
Task/Container	Application stage may fail	Retry/re-execution
Network	Communication and throughput affected	Redundancy + monitoring

Practical Interview Insight

Failure handling should combine redundancy, replication, high availability, monitoring, retries and tested recovery procedures.

Interview Answer Flowchart



Question 9 – Secure Data Ingestion Pipeline

Interview Objective

Provide a clear, concise and technically accurate interview response, with architecture-level reasoning where appropriate.

Answer

- A secure ingestion pipeline should protect data while it is moving from source systems to the data platform and while it is stored.
- Use encrypted transport such as TLS for network communication between ingestion components where supported.
- Authenticate users, services and applications, and authorize access using least-privilege policies.
- Validate incoming data and control which sources are allowed to write to the platform.
- Use audit logging and provenance to record ingestion activity.
- Sensitive data should receive appropriate encryption, access control and retention policies.
- Failure handling should avoid accidental duplicate loads and should preserve traceability.

Key Points

- A secure ingestion pipeline should protect data while it is moving from source systems to the data platform and while it is stored.
- Use encrypted transport such as TLS for network communication between ingestion components where supported.
- Authenticate users, services and applications, and authorize access using least-privilege policies.
- Validate incoming data and control which sources are allowed to write to the platform.

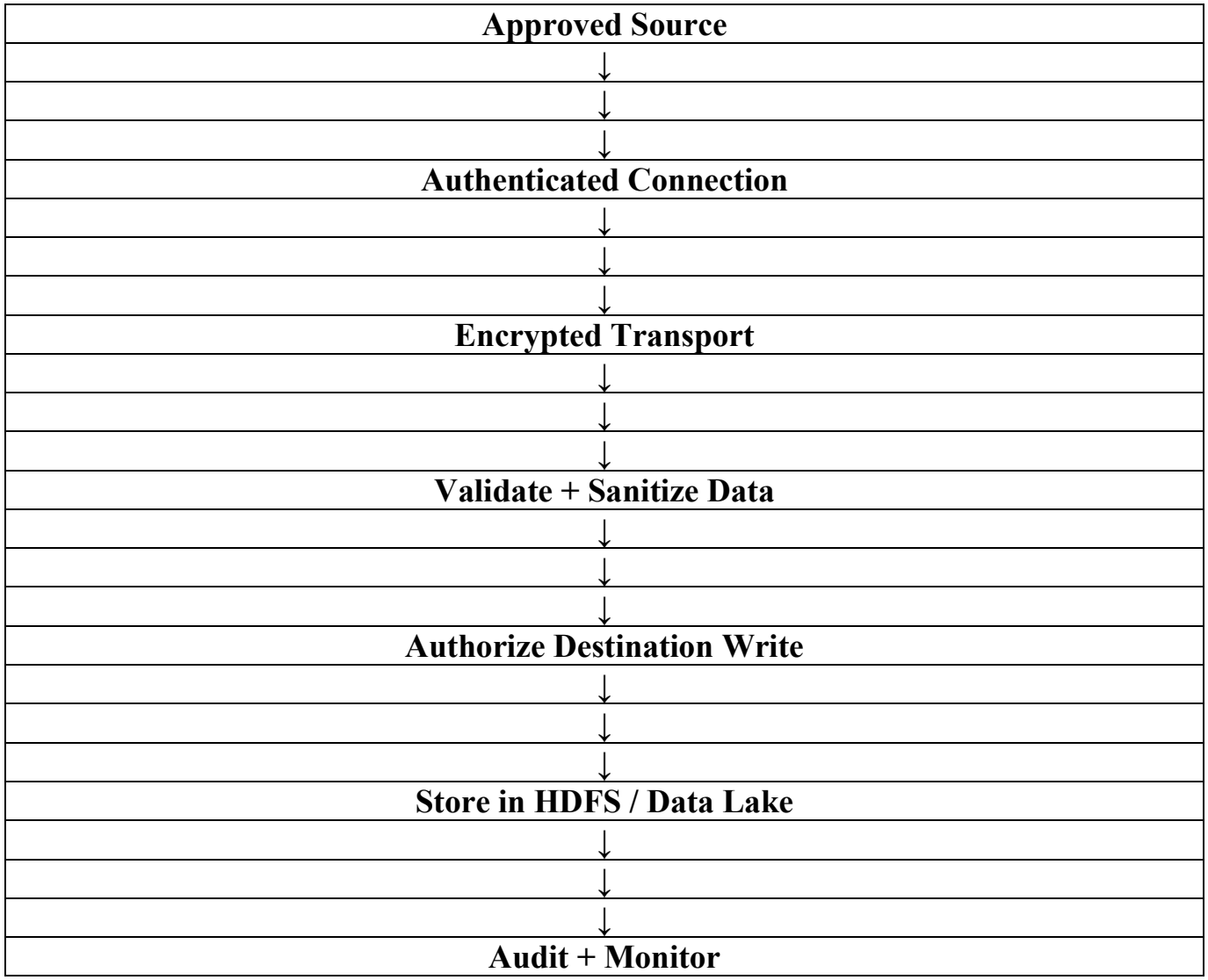
Technical Comparison / Structure

Security Layer	Design Measure
Transport	TLS/encrypted communication
Identity	Strong authentication for users/services
Authorization	Role/least-privilege access
Data	Encryption and sensitive-data controls
Validation	Schema and content validation
Audit	Access, ingestion and provenance logs
Reliability	Retry-safe/idempotent ingestion

Practical Interview Insight

Security should be designed end-to-end: authenticate, authorize, encrypt, validate, audit and monitor the data flow.

Interview Answer Flowchart



Question 10 – Explaining a Scalable Hadoop System to a Non-Technical Manager

Interview Objective

Provide a clear, concise and technically accurate interview response, with architecture-level reasoning where appropriate.

Answer

- A scalable Hadoop system can be explained as a group of computers working together as one large data-processing platform.
- HDFS stores large datasets by splitting files into blocks and distributing them across multiple machines.
- Replication keeps additional copies so that a machine failure does not necessarily stop access to important data.
- YARN acts like a resource manager that decides which jobs receive computing resources.
- MapReduce or other processing engines divide large tasks into smaller pieces that can run in parallel.
- When data grows, additional machines can be added to increase storage and processing capacity.
- The business value is the ability to process large volumes of data reliably without depending on a single powerful machine.

Key Points

- A scalable Hadoop system can be explained as a group of computers working together as one large data-processing platform.
- HDFS stores large datasets by splitting files into blocks and distributing them across multiple machines.
- Replication keeps additional copies so that a machine failure does not necessarily stop access to important data.
- YARN acts like a resource manager that decides which jobs receive computing resources.

Technical Comparison / Structure

Technical Concept	Manager-Friendly Explanation
HDFS	A distributed storage system that spreads large files across many machines.
Replication	Backup-like copies that keep data available when machines fail.
YARN	A resource manager that decides how computing power is shared.
MapReduce	A method that splits a large job into

	smaller parallel tasks.
Scalability	Add more machines as data and workload increase.
Fault Tolerance	The system continues operating despite individual failures.

Practical Interview Insight

For a manager, explain the system in terms of business outcomes: scale, reliability, capacity expansion, parallel processing and reduced dependence on a single machine.

Interview Answer Flowchart

Business Data Grows
↓
↓
↓
Add Storage / Compute Nodes
↓
↓
↓
HDFS Distributes Data
↓
↓
↓
YARN Allocates Resources
↓
↓
↓
Parallel Processing
↓
↓
↓
Reliable Results
↓
↓
↓
Scale Further When Required

Mock Interview Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Technical Knowledge	25%	Shows deep and accurate technical understanding	Good technical understanding	Basic understanding	Weak understanding
Problem Solving	20%	Responds with strong architecture-level reasoning	Reasonable reasoning	Basic reasoning	Weak reasoning
Communication	20%	Answers are confident, precise, and professional	Mostly clear communication	Moderate clarity	Weak communication
Practical Awareness	20%	Connects concepts to real-world implementation well	Some practical relevance	Limited practical relevance	No practical linkage
Confidence and Professionalism	15%	Highly confident and professional	Generally professional	Moderately professional	Low confidence and professionalism

Submission Checklist

- All ten interview questions are answered clearly and concisely.
- HDFS, MapReduce, YARN, NAS, SAN, ETL and Apache NiFi concepts are explained accurately.
- Architecture-level reasoning is included where appropriate.
- Each question contains a structured answer, a summary table and a vertical flowchart at the end.
- Every question begins on a separate page.
- All side headings are bold and underlined, with Times New Roman 14-point formatting throughout.